INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

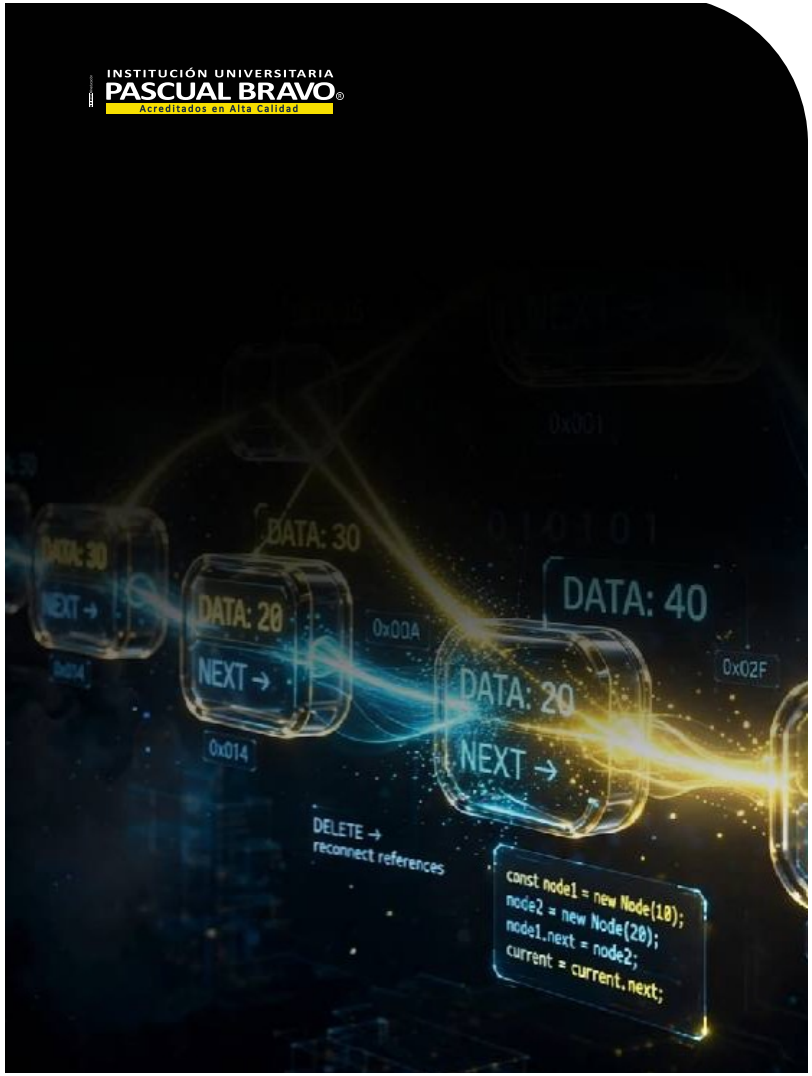
Formar
Servir
Cuidar

Lista enlazada



Las **Lista enlazada** son una estructura de datos en la que cada elemento de la lista contiene un enlace al siguiente elemento

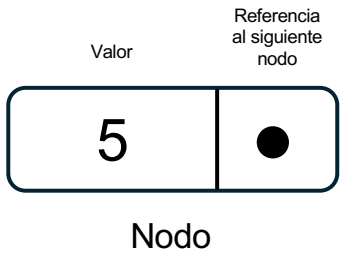
https://www.freecodecamp.org/news/what-is-a-linked-list-types-and-examples/

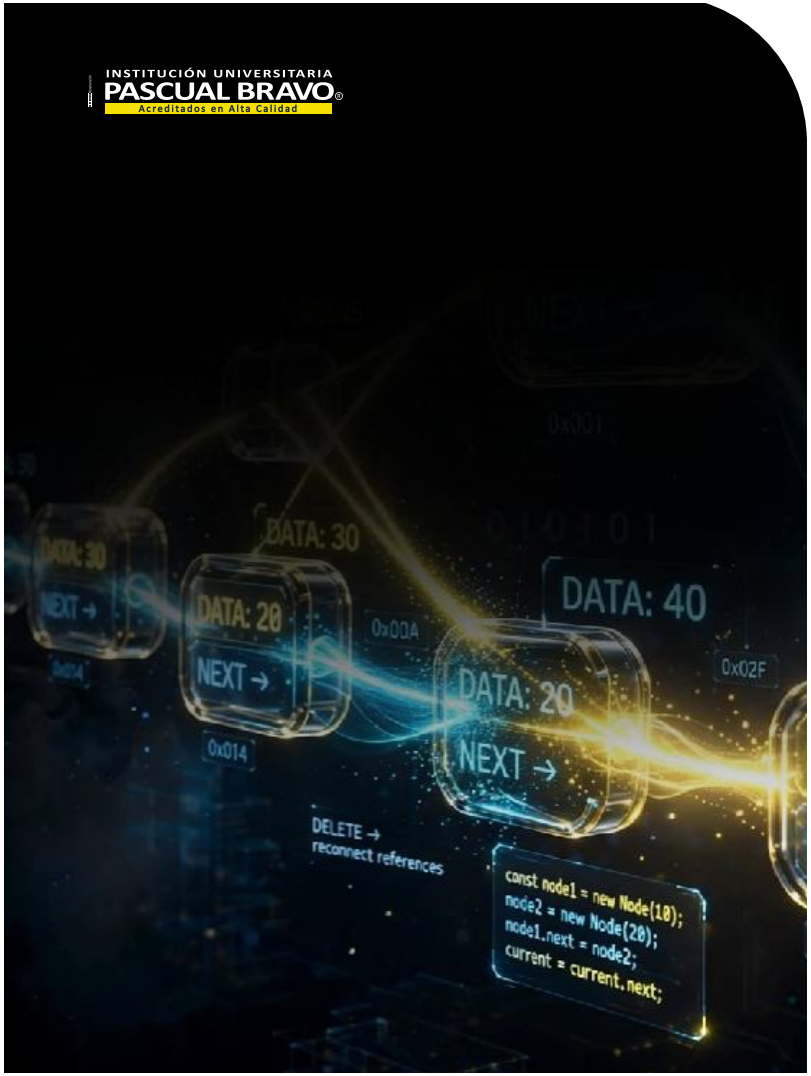


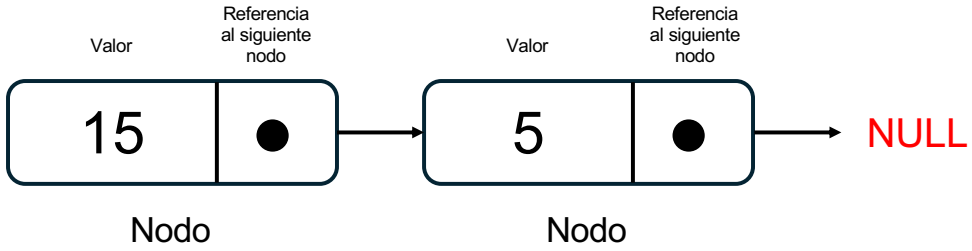
Formar
Servir
Cuidar

Cada elemento de una **lista enlazada** se llama nodo y cada nodo guarda dos cosas

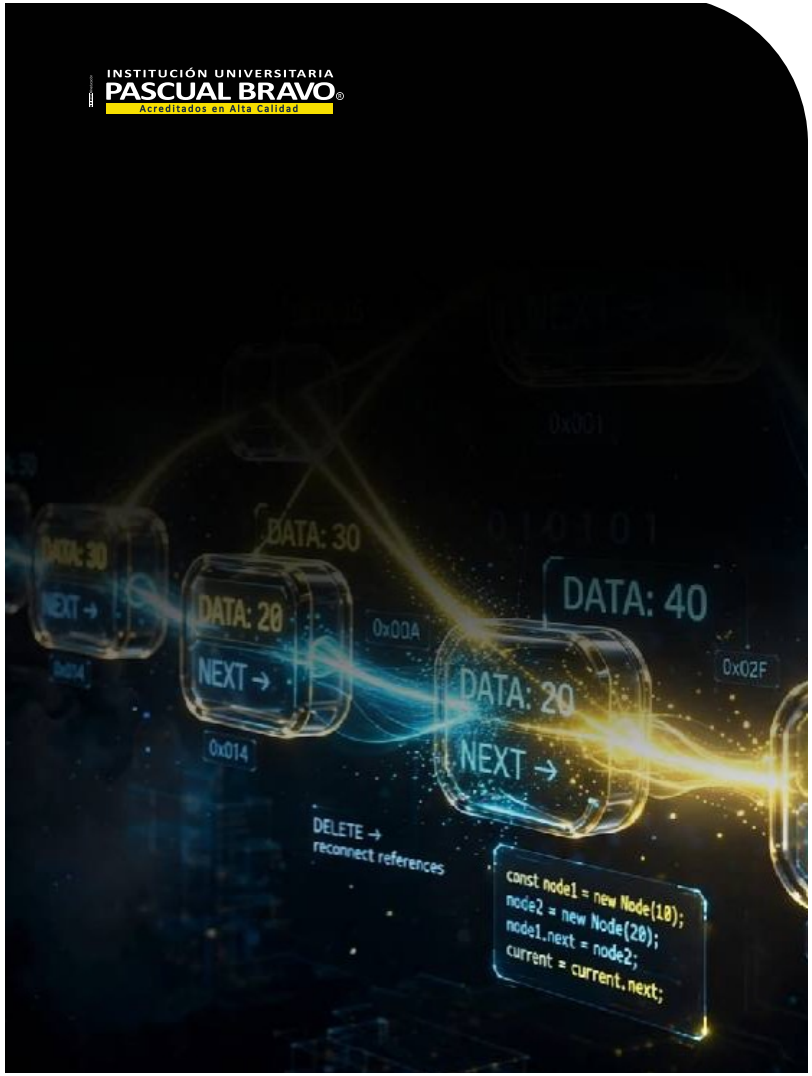
1. Valor
2. La referencia al **siguiente** nodo







El último apunta a **null** y allí termina la lista

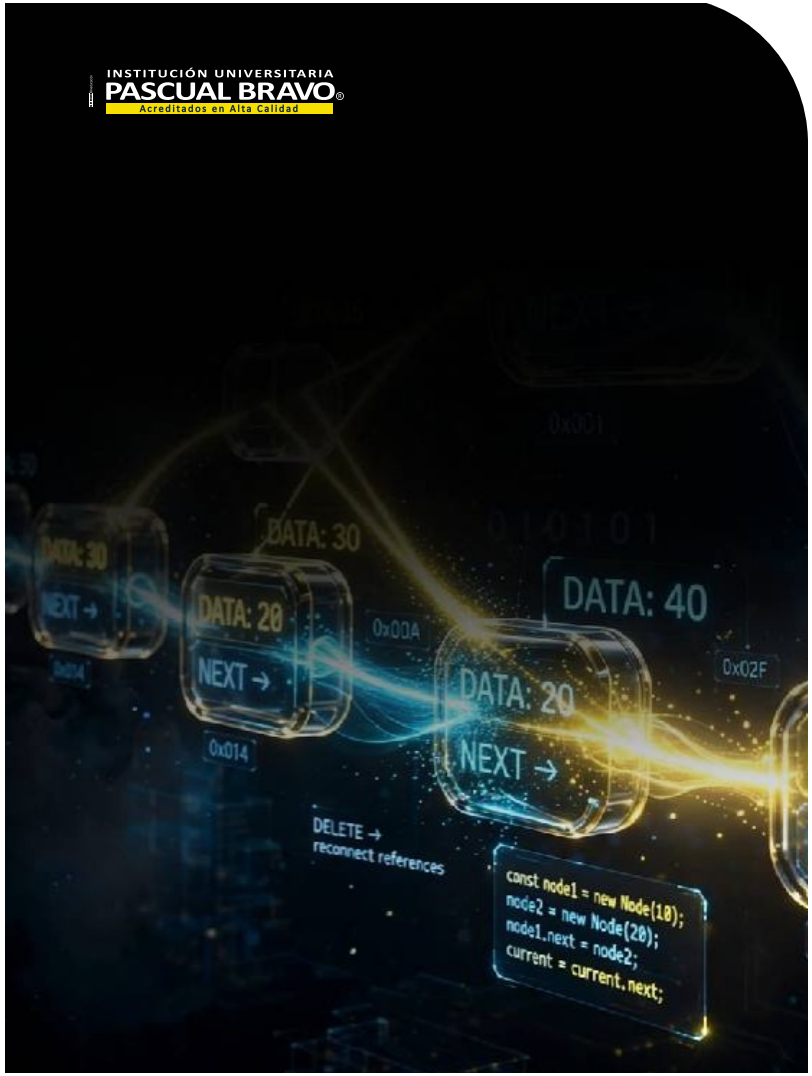


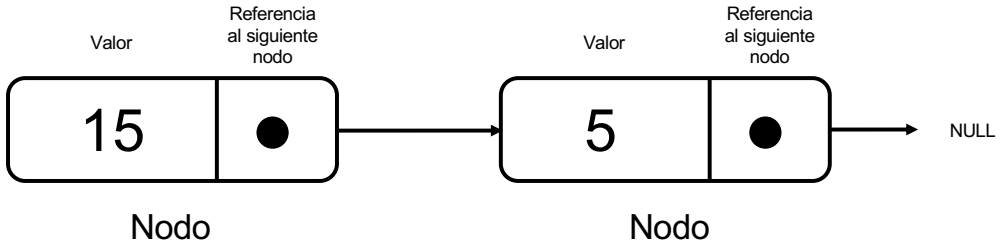
Formar
Servir
Cuidar

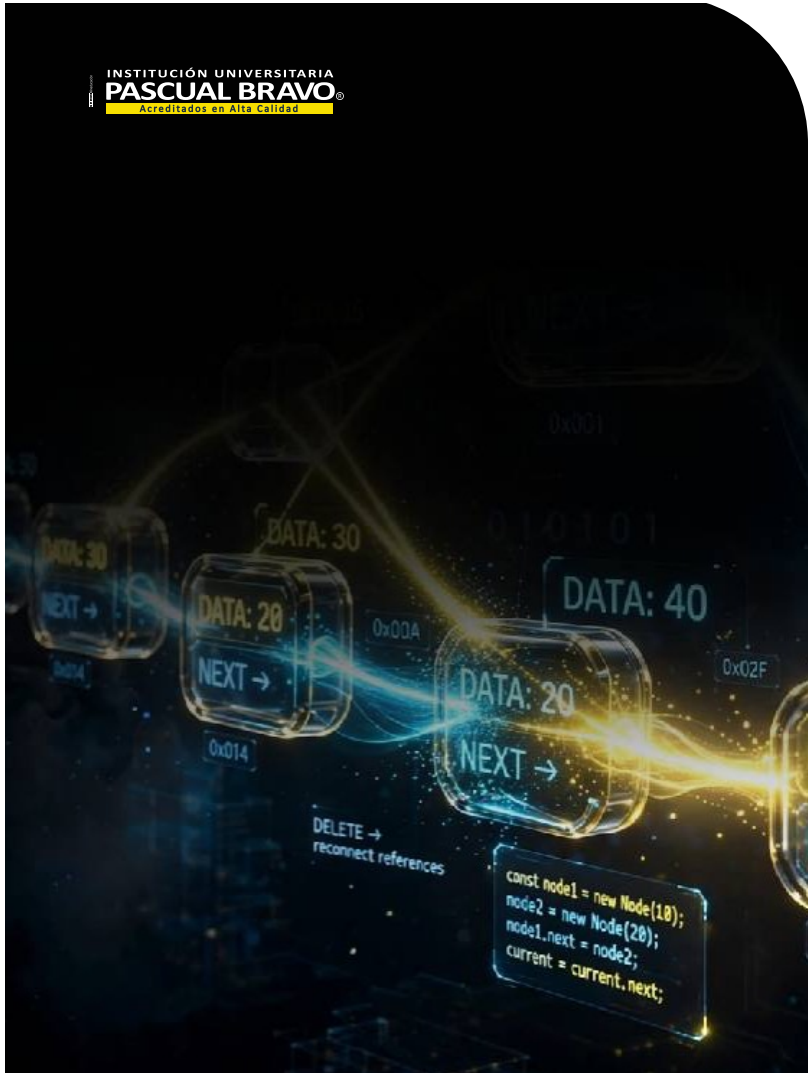
¿Como agregar un nuevo nodo **entre dos nodos** existentes?

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

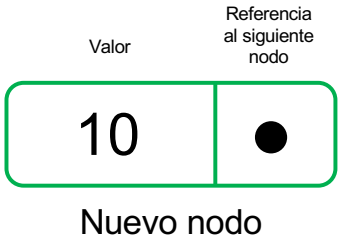
Formar
Servir
Cuidar



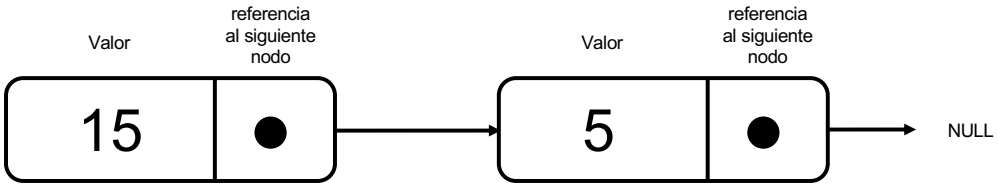




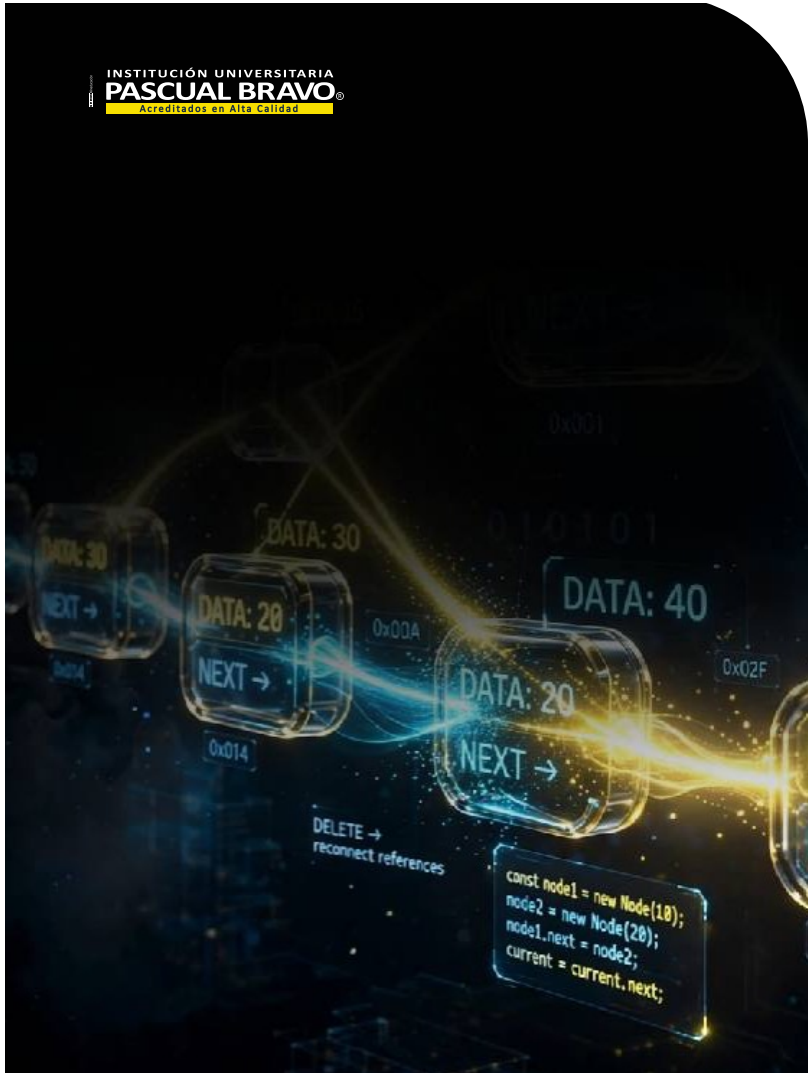
Formar
Servir
Cuidar



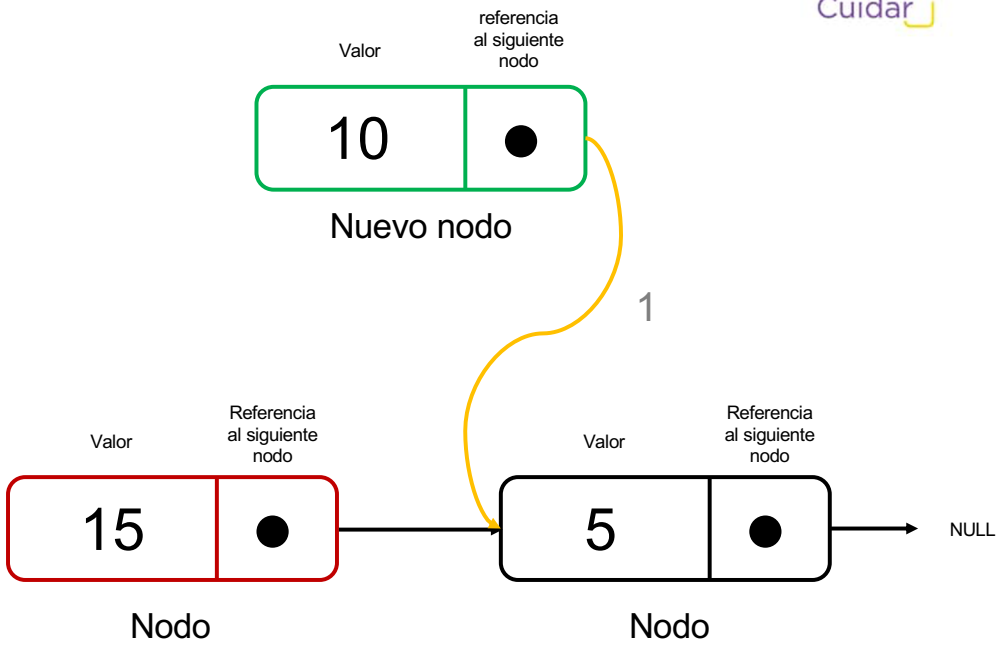
Nuevo nodo

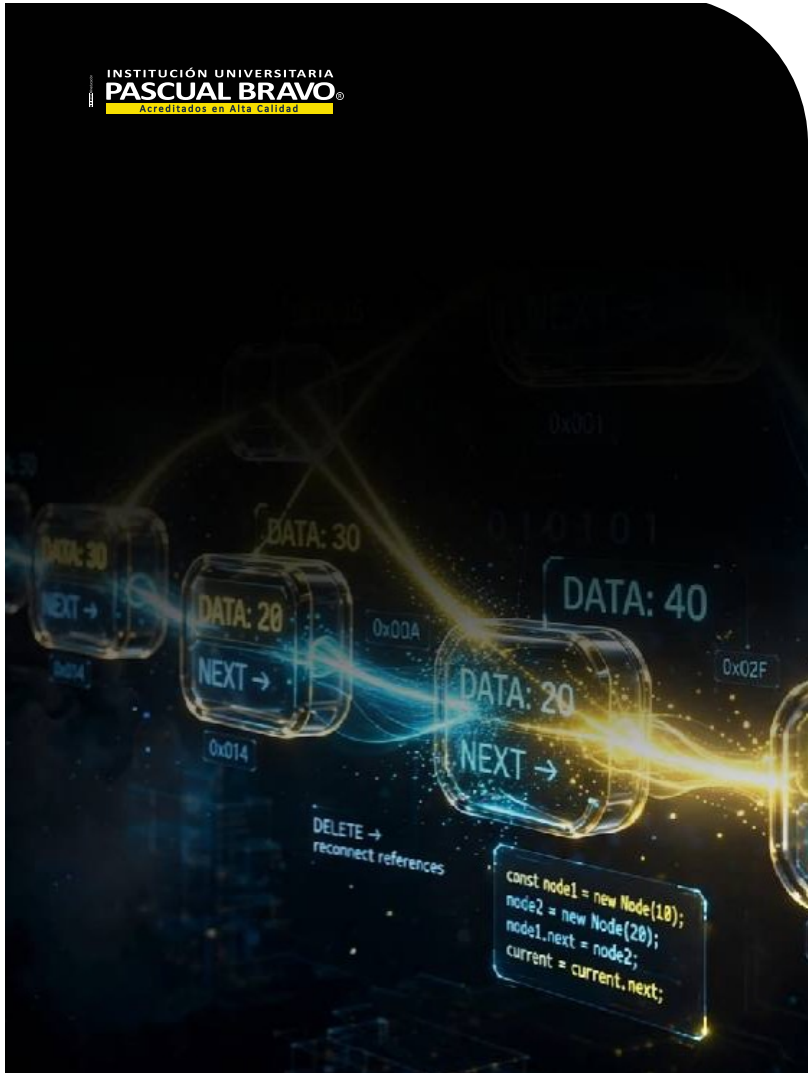


Nodo Nodo

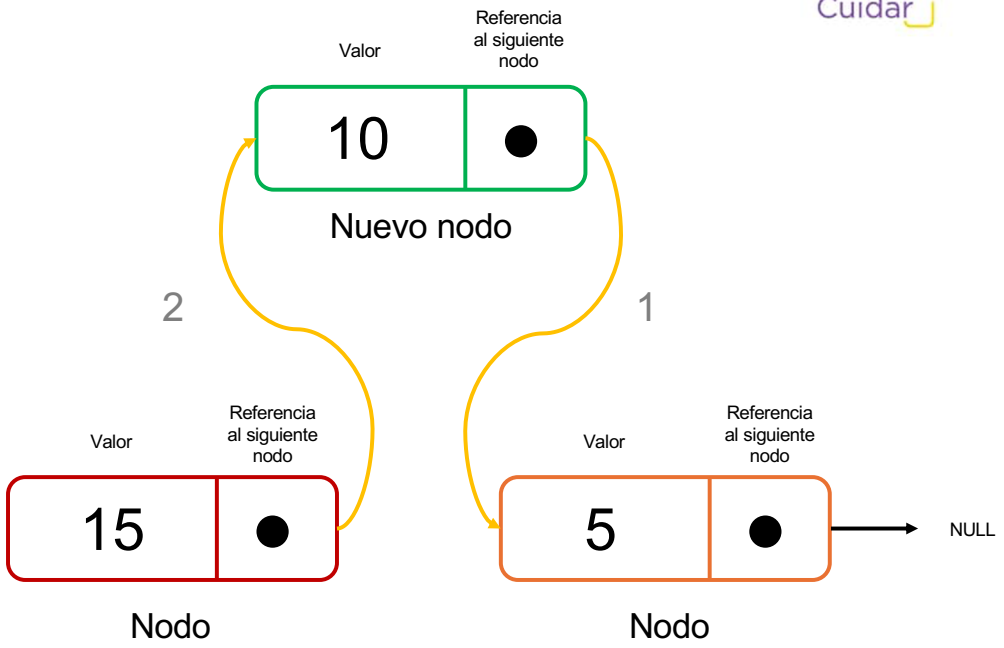


Formar
Servir
Cuidar



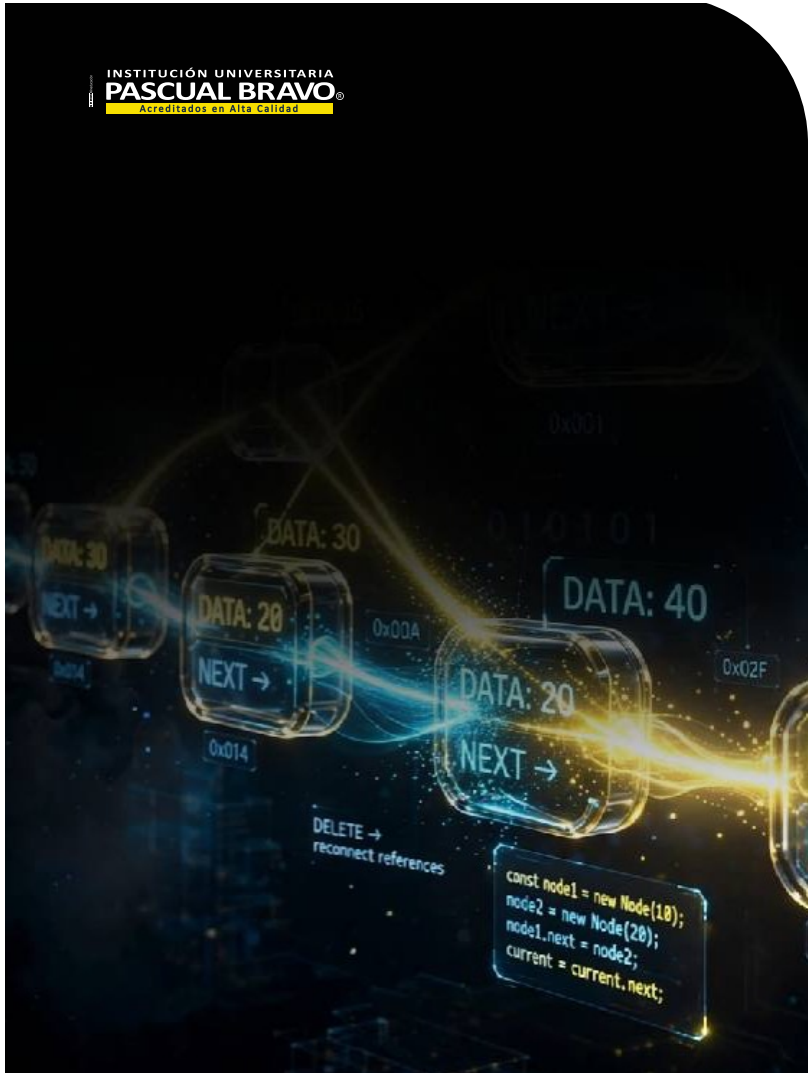


Formar
Servir
Cuidar



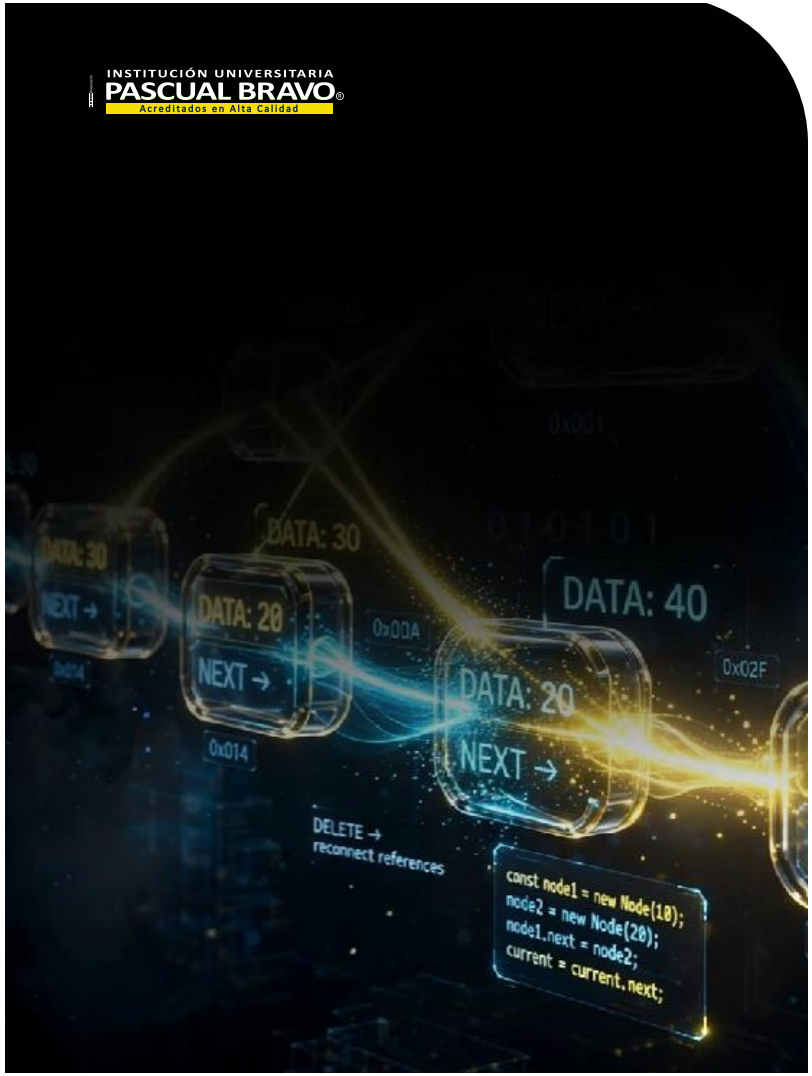
Solo se cambian dos referencias

1. El nodo nuevo apunta al que seguía
2. El nodo anterior apunta al nuevo



Formar
Servir
Cuidar

Agregar un nodo
en una lista con **n número de elementos** tarda lo mismo que en una de 2,5,10 ó n elementos
Solo si sabemos donde agregarlo



Formar Servir Cuidar

Pero si quiero el valor del nodo 100 se debe ir **nodo** por **nodo** desde el inicio siguiendo la referencia.

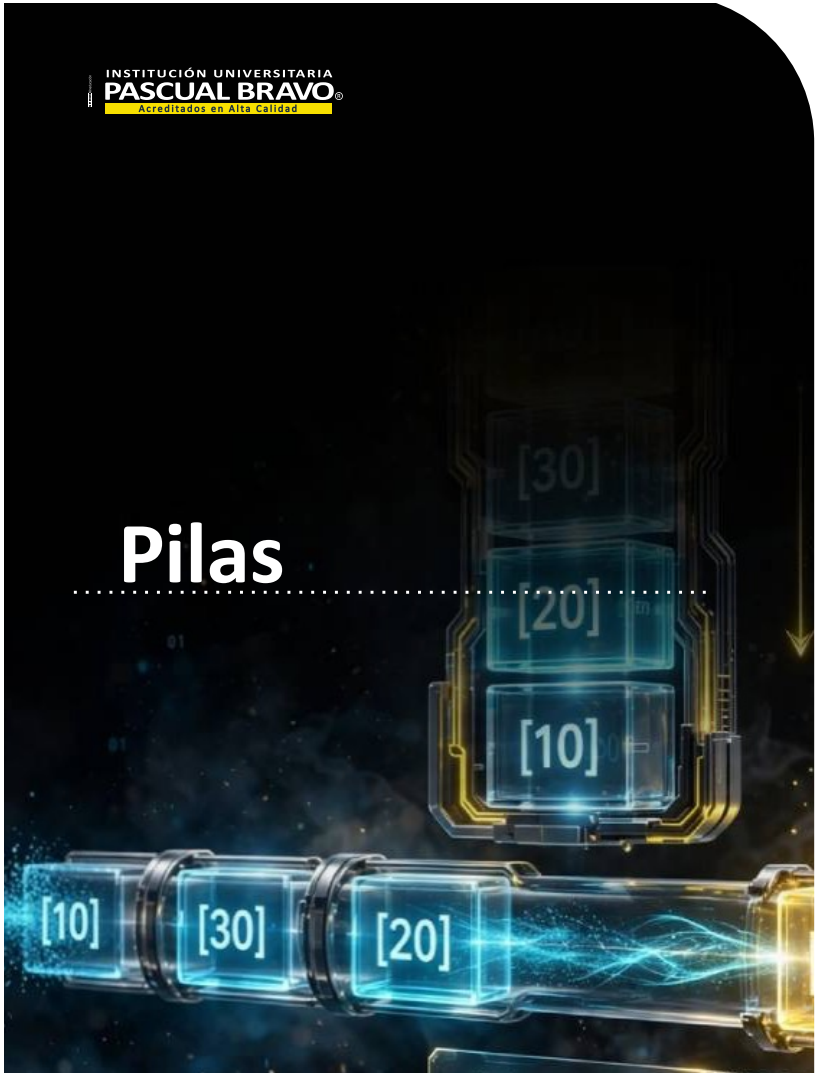
No hay atajo como en el array

Constante O(1)

dato[2]

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Pilas



Formar
Servir
Cuidar

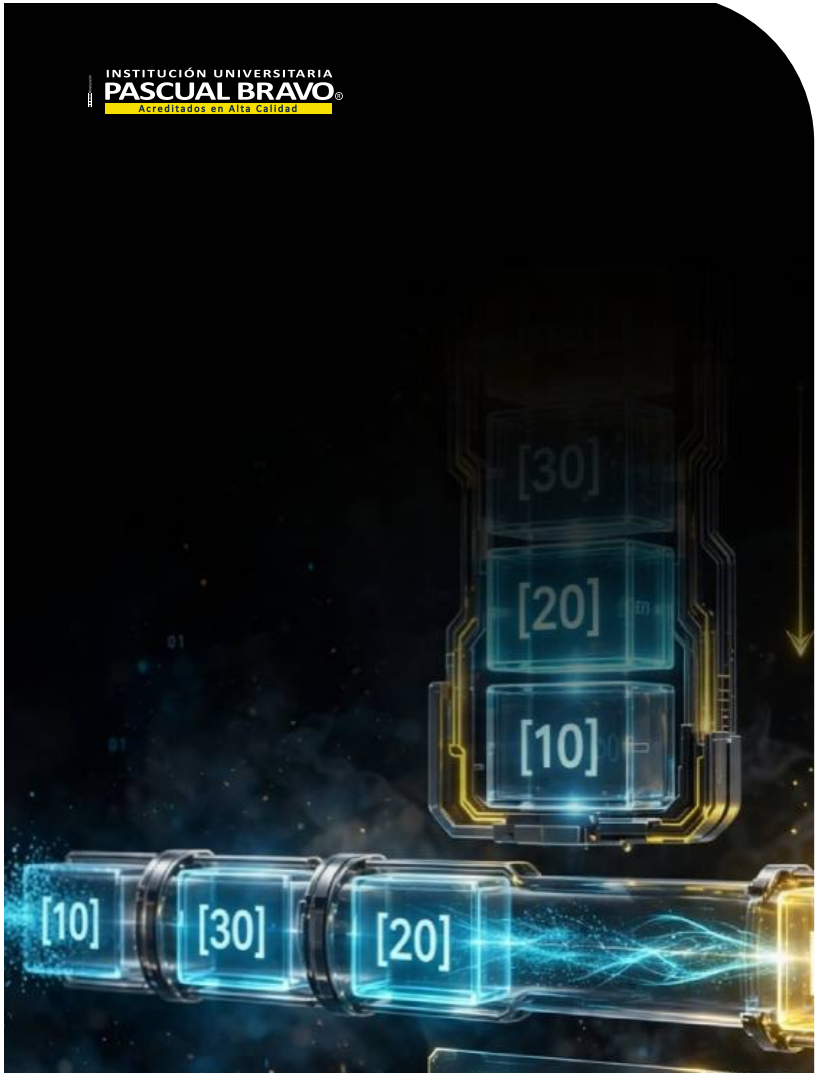
LIFO — Last In, First Out
El último en entrar, es el primero en salir

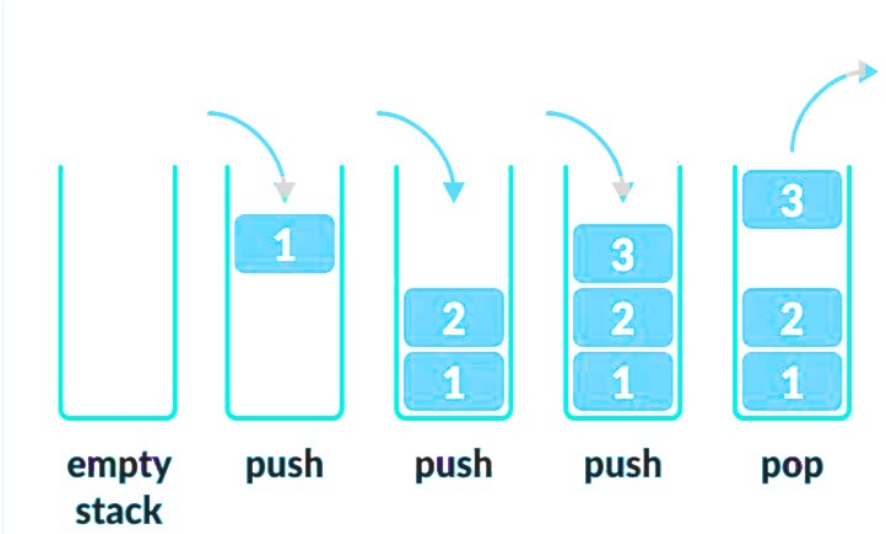


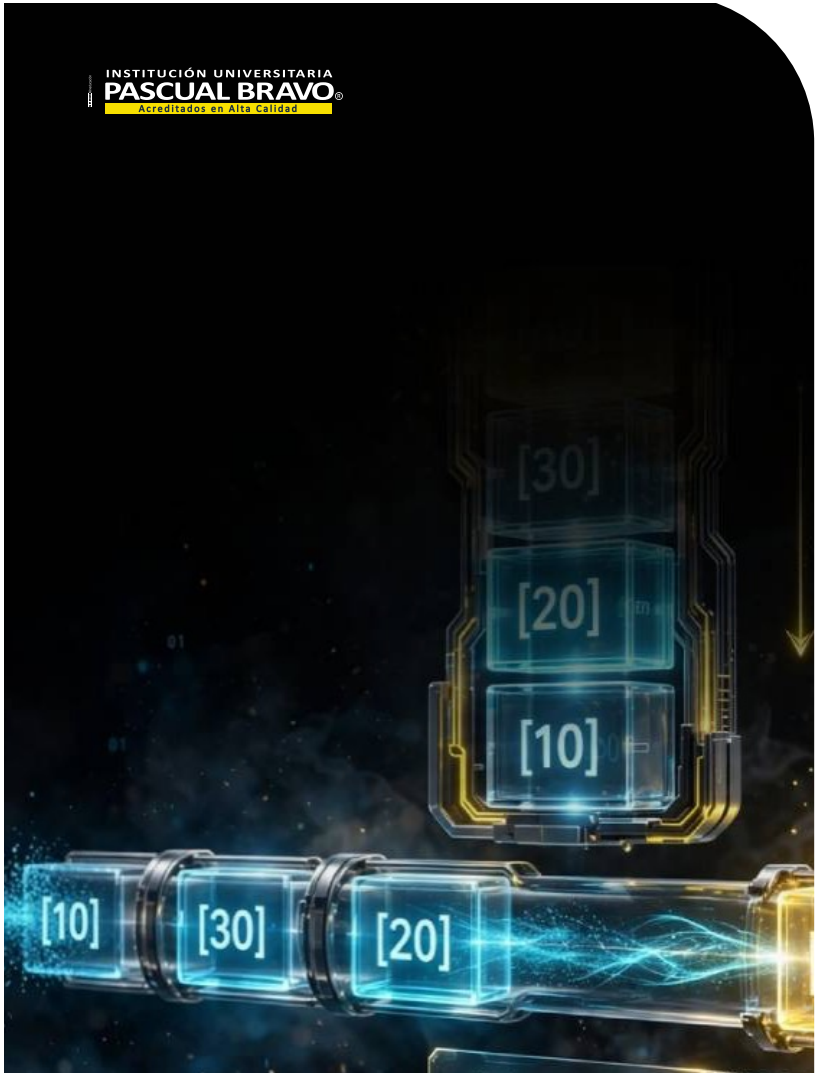
https://amsoft.medium.com/dominando-las-estructuras-de-datos-en-java-pilas-y-colas-09a4279264b1

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar







Formar
Servir
Cuidar

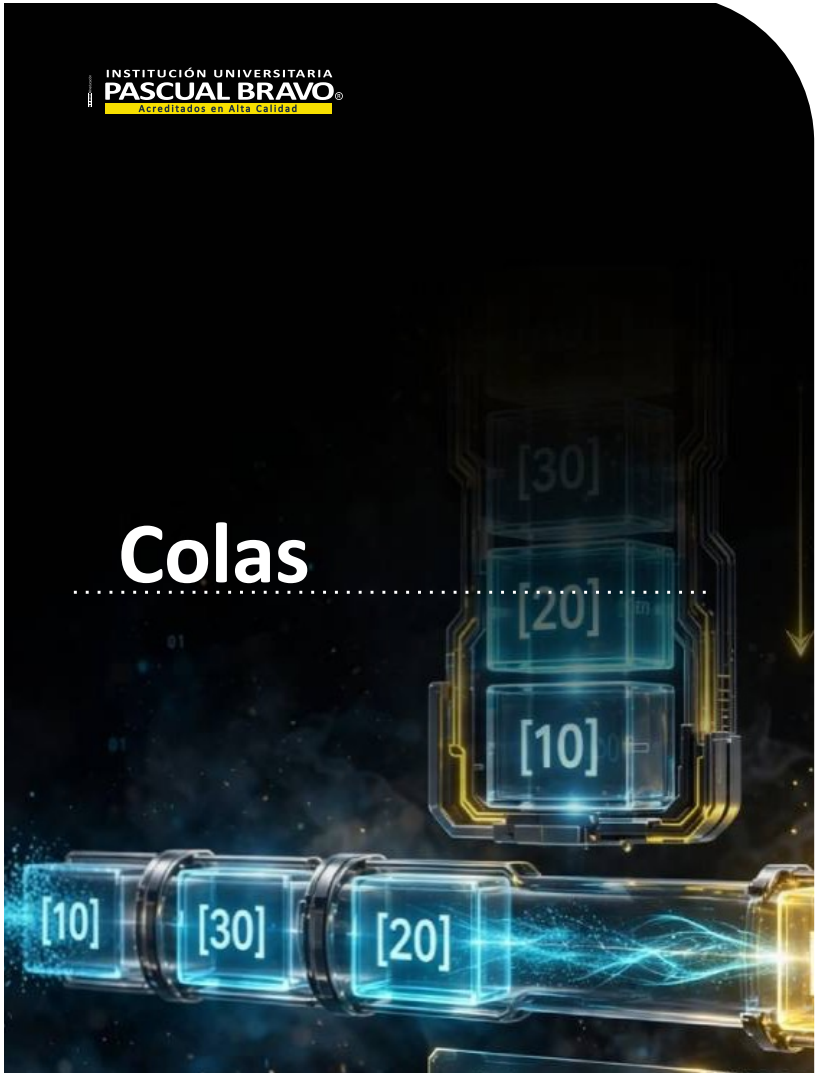
Ejemplos de pilas:

Historial de navegación
Control+z teclado
Libros apilados
Entre otros

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

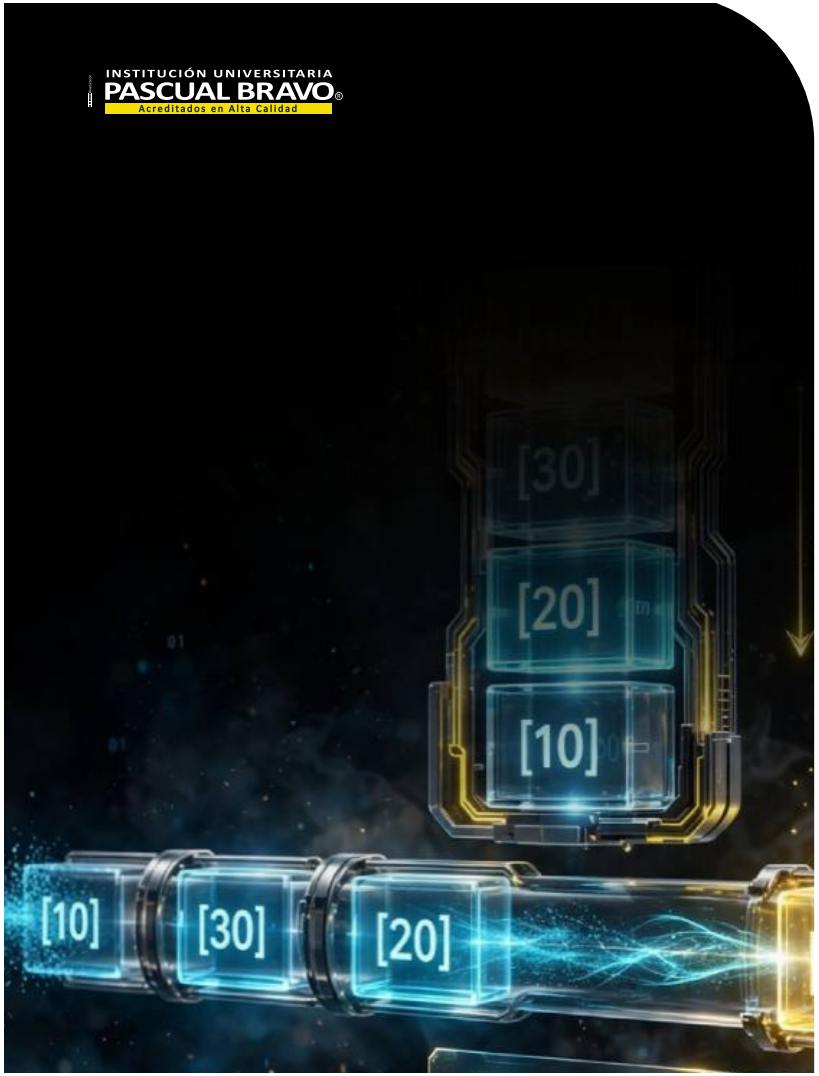
Colas



FIFO = First In, First Out
El primer elemento que entra es el primero que sale.

2	9	30	40	8	5

Siempre se respeta el órden de llegada



Ejemplos colas:

Impresora
Fila en un banco
Las peticiones en los servidores
Peaje
Fila del Supermercado
Entre otros

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

Heap

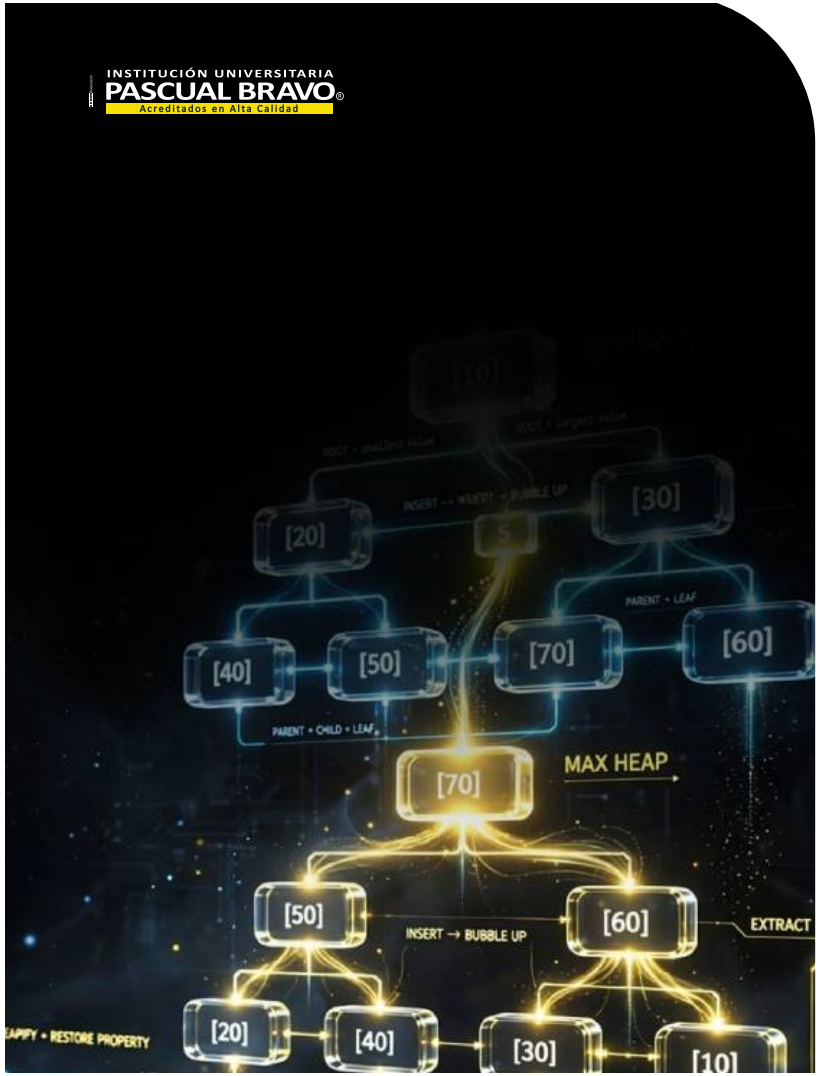


El elemento de mayor prioridad siempre esta en la raíz independiente del orden que ingresó

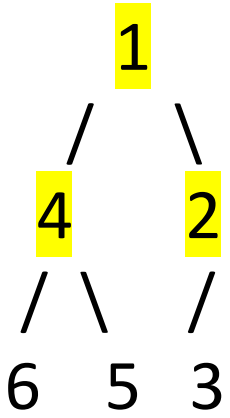
Cuando se extraen elementos el Heap se reorganiza para cumplir la regla

https://www.npmjs.com/package/heap-js

https://docs.python.org/es/3.14/library/heapq.html



Formar
Servir
Cuidar



Cada nodo padre tiene una prioridad menor o igual que sus hijos.

La regla se cumple:
1 < 4
1 < 2

4 < 6
4 < 5

2 < 3

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

Grafos



La Teoría de Grafos es una forma de representar cosas y las conexiones que existen entre ellas.

Consiste en un conjunto de nodos conectados por enlaces

https://www.youtube.com/watch?v=9dlDO7crQF0
https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar



¿Por qué son importantes en programación?

Los grafos permiten resolver problemas como:

- Encontrar caminos.
- Buscar conexiones.
- Encontrar la ruta más corta.
- Saber quién está conectado con quién.
- Analizar redes sociales.
- Crear sistemas de navegación.
- Analizar redes de computadores.
- Crear mapas.
- Resolver problemas de videojuegos.
- Organizar dependencias entre tareas.



Formar
Servir
Cuidar

Un grafo tiene principalmente

Nodos: Son las cosas que queremos representar.

Ejemplo:

- Medellín
- Bogotá
- Cali
- Cartagena

(Cada uno representa un nodo)



Formar
Servir
Cuidar

Aristas o conexiones: Son las relaciones entre los nodos.

Ejemplo: Medellín —— Bogotá



Podemos representarlo de la siguiente manera:

conexiones = {
"Bogotá" -> ["Medellín", "Bucaramanga"]
"Medellín" -> ["Bogotá", "Pereira", "Cartagena"]
"Bucaramanga" -> ["Bogotá"]
"Pereira" -> ["Medellín"]
"Cartagena" -> ["Medellín"]
}



Formar Servir Cuidar

Las distancias son:

Origen	Destino	Distancia
A	B	4 km
A	C	2 km
C	D	3 km
D	B	1 km

Queremos ir de A hasta B.
¿Qué caminos existen?
Camino 1: A → B // Distancia: 4 km
Camino 2: A → C → D → B // Distancia: 2 + 3 + 1 = 6 km

La ruta más corta es: A → B



Formar Servir Cuidar

Mesa de ayuda

Los nodos pueden ser:
Usuario
Ticket
Computador
Aplicación
Servidor
API
Base de datos

Las relaciones:
Usuario —crea—> Ticket
Ticket —afecta—> Aplicación
Aplicación —depende—> API
API —depende—> Base de datos

Si 20 tickets están relacionados con la ERP y todos terminan dependiendo de DB-01, el sistema puede detectar:



Formar Servir Cuidar

Detección de fraude

Los nodos pueden ser:
Cliente
Cuenta
Tarjeta
Dispositivo
IP
Transacción

Las relaciones:
Cliente → Cuenta
Cuenta → Tarjeta
Tarjeta → Transacción
Transacción → IP
Transacción → Dispositivo

El sistema puede detectar que varias cuentas están relacionadas con el mismo dispositivo.

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

Appresso (Tu café, a un tap)

Contexto:
Appresso necesita desarrollar un sistema para gestionar la atención de clientes y planificar sus entregas. El sistema deberá organizar las solicitudes mediante pilas y colas, representar las ciudades mediante grafos, calcular rutas y distancias, y estimar el consumo y costo de gasolina.

Objetivo
Desarrollar un sistema que integre pilas, colas y grafos (Dijkstra) para administrar clientes y entregas. El sistema deberá gestionar solicitudes según su orden de llegada, controlar operaciones recientes, calcular rutas entre ciudades, determinar distancias recorridas y estimar el gasto de gasolina de cada entrega.

Lo esperado
Implementar un sistema funcional que permita registrar y atender clientes utilizando pilas y colas, representar rutas mediante grafos y calcular distancias entre ciudades. Finalmente, deberá estimar los litros de gasolina necesarios y el costo asociado según el recorrido realizado.

Appresso (Tu café, a un tap)

Escenarios y casos de uso para pilas y colas

Escenario	Cola	Pila
Banco	Turnos de clientes	Operaciones recientes
Restaurante	Pedidos pendientes	Deshacer modificación de pedido
Hospital	Atención de pacientes	Historial de acciones
Call center	Llamadas pendientes	Últimas llamadas gestionadas
Supermercado	Clientes en cajas	Historial de productos escaneados
Soporte técnico	Tickets pendientes	Acciones recientes del técnico
Videojuego	Jugadores esperando partida	Historial de movimientos